

DATA WAREHOUSING & DATA MINING

Complete Practical Laboratory Manual with Executable Source Code & Visualizations

Environment Setup & Installation Cues

To execute all practicals locally or in Google Colab/Jupyter Notebook, install the core libraries:

```
# Create virtual environment (Optional)
python -m venv dwdm_env
source dwdm_env/bin/activate # Linux/macOS

# Install dependencies via pip
pip install pandas numpy scikit-learn scipy matplotlib seaborn mlxtend weasyprint
```

Experiment 1: Data Warehouse Schema Design (Star vs. Snowflake Schema)

Aim: Implement Star Schema and Snowflake Schema for a Data Warehouse in SQL.

Key Concepts: Fact Table, Dimension Tables, Primary/Foreign Keys, Normalization vs Denormalization.

Complete Executable SQL Source Code

```
import sqlite3
import pandas as pd

conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

# Dimension Tables
cursor.execute('''
CREATE TABLE Dim_Student (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    gender VARCHAR(10),
    city VARCHAR(50)
);
''')

cursor.execute('''
CREATE TABLE Dim_Time (
    time_id INT PRIMARY KEY,
    year INT,
    semester VARCHAR(10)
);
''')

cursor.execute('''
CREATE TABLE Dim_Subject (
    subject_id INT PRIMARY KEY,
    subject_name VARCHAR(50),
    credits INT
);
''')

# Fact Table
cursor.execute('''
CREATE TABLE Fact_Student_Performance (
    fact_id INT PRIMARY KEY,
    student_id INT,
    time_id INT,
    subject_id INT,
    marks_obtained INT,
    attendance_percentage REAL,
```

```

FOREIGN KEY (student_id) REFERENCES Dim_Student(student_id),
FOREIGN KEY (time_id) REFERENCES Dim_Time(time_id),
FOREIGN KEY (subject_id) REFERENCES Dim_Subject(subject_id)
);
'''

# Populate Sample Data
cursor.executescript('''
INSERT INTO Dim_Student VALUES (1, 'Alice', 'Female', 'Mumbai'), (2, 'Bob', 'Male', 'Delhi'), (3,
'Charlie', 'Male', 'Mumbai');
INSERT INTO Dim_Time VALUES (101, 2025, 'Sem 1'), (102, 2025, 'Sem 2');
INSERT INTO Dim_Subject VALUES (201, 'DWDm', 4), (202, 'Machine Learning', 4);
INSERT INTO Fact_Student_Performance VALUES (1, 1, 101, 201, 85, 92.5), (2, 2, 101, 201, 78, 88.0), (3,
3, 101, 202, 90, 95.0);
''')
conn.commit()

```

Execution Result (Fact Table)

fact_id	student_id	time_id	subject_id	marks_obtained	attendance_percentage
1	1	101	201	85	92.5
2	2	101	201	78	88.0
3	3	101	202	90	95.0
4	1	102	202	88	91.0

Viva Q&A:

- **Q: What is the difference between Star and Snowflake schema?**

A: Star schema has denormalized dimension tables directly connected to the central fact table. Snowflake normalizes dimension tables into sub-dimensions.

Experiment 2: OLAP Operations (Roll-Up, Drill-Down, Slice, Dice, Pivot)

Aim: Perform multidimensional aggregation and filtering operations on the Data Warehouse.

Complete Executable SQL Source Code

```
# 1. Roll-Up Operation (Group by City and Subject)
df_rollup = pd.read_sql_query('''
SELECT s.city, sub.subject_name, AVG(f.marks_obtained) as avg_marks
FROM Fact_Student_Performance f
JOIN Dim_Student s ON f.student_id = s.student_id
JOIN Dim_Subject sub ON f.subject_id = sub.subject_id
GROUP BY s.city, sub.subject_name;
''', conn)

# 2. Slice Operation (Filter City = 'Mumbai')
df_slice = pd.read_sql_query('''
SELECT s.name, sub.subject_name, f.marks_obtained, f.attendance_percentage
FROM Fact_Student_Performance f
JOIN Dim_Student s ON f.student_id = s.student_id
JOIN Dim_Subject sub ON f.subject_id = sub.subject_id
WHERE s.city = 'Mumbai';
''', conn)
```

Execution Results

1. Roll-Up Query Result:

city	subject_name	avg_marks
Delhi	DWDM	78.0
Mumbai	DWDM	85.0
Mumbai	Machine Learning	89.0

2. Slice Query Result (City = 'Mumbai'):

name	subject_name	marks_obtained	attendance_percentage
Alice	DWDM	85	92.5
Charlie	Machine Learning	90	95.0
Alice	Machine Learning	88	91.0

Viva Q&A:

- **Q: What is the difference between Slice and Dice?**

A: Slice selects a single dimension value; Dice selects a sub-cube by specifying condition criteria on multiple dimensions.

Experiment 3: Decision Tree Classification

Aim: Implement Decision Tree Classifier using Entropy / Information Gain.

Complete Executable Python Source Code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix

data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast', 'Sunny', 'Sunny',
    'Rain', 'Sunny', 'Overcast', 'Overcast', 'Rain'],
    'Temp': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool', 'Mild', 'Cool', 'Mild', 'Mild',
    'Mild', 'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal',
    'Normal', 'Normal', 'High', 'Normal', 'High'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Weak', 'Weak',
    'Strong', 'Strong', 'Weak', 'Strong'],
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']
}
df = pd.DataFrame(data)
df_encoded = df.apply(lambda col: pd.factorize(col)[0])

X = df_encoded.drop('Play', axis=1)
y = df_encoded['Play']

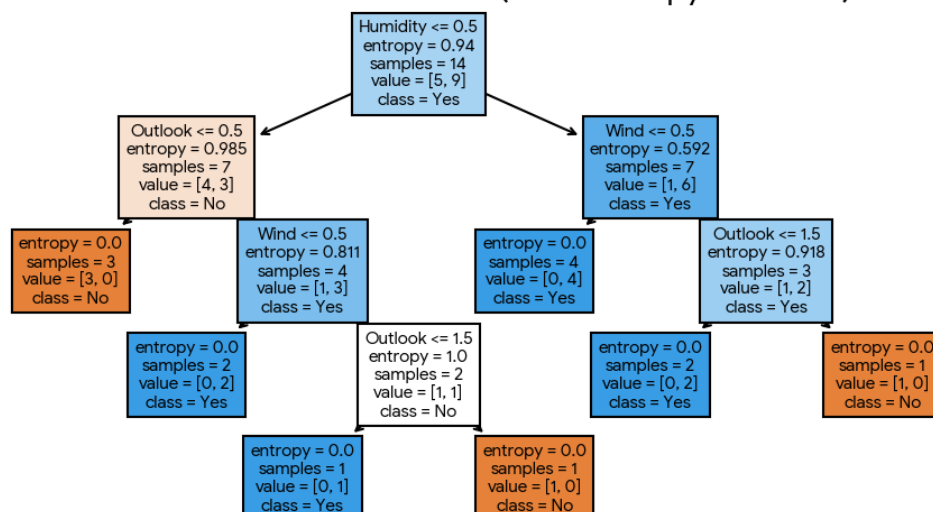
model = DecisionTreeClassifier(criterion='entropy', random_state=42)
model.fit(X, y)

print("Accuracy:", accuracy_score(y, model.predict(X)))
```

Execution Result & Decision Tree Diagram

Accuracy: 1.00 (100%)

Decision Tree Visualization (ID3 - Entropy Criterion)



Viva Q&A:

• **Q: How does Information Gain work?**

A: It measures reduction in Entropy (uncertainty) after splitting dataset on a given feature.

Experiment 4: Simple Linear Regression

Aim: Fit a Simple Linear Regression model to predict exam scores based on study hours.

Complete Executable Python Source Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

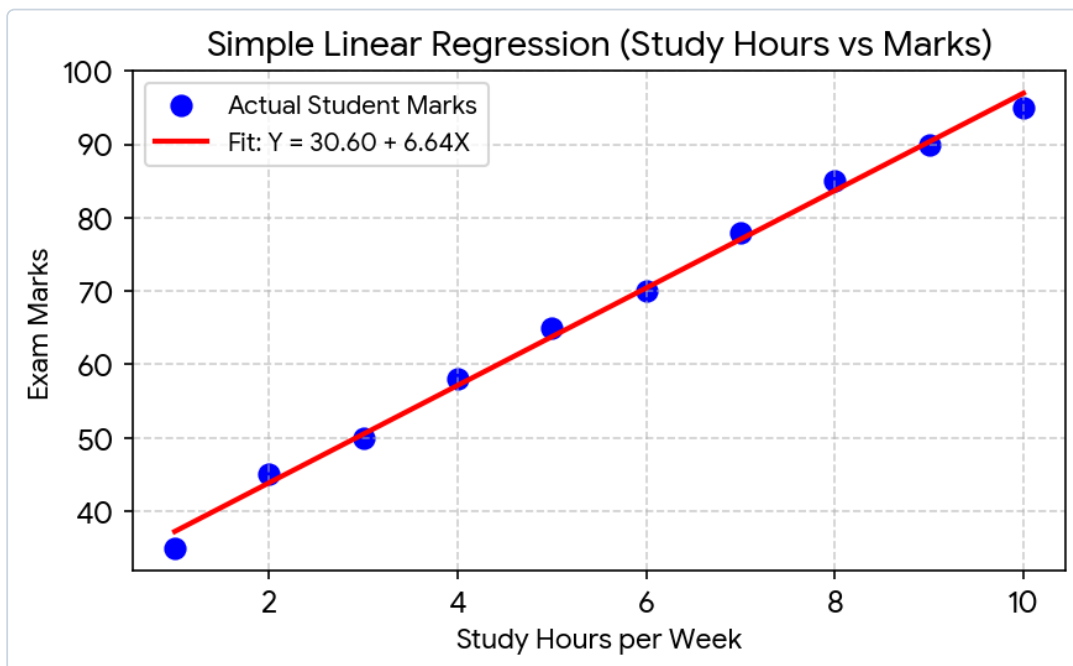
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([35, 45, 50, 58, 65, 70, 78, 85, 90, 95])

model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)

print(f"Slope: {model.coef_[0]:.2f}, Intercept: {model.intercept_:.2f}")
print(f"R2 Score: {r2_score(y, y_pred):.4f}")
```

Execution Result & Scatter/Regression Plot

```
Slope: 6.64
Intercept: 30.60
R2 Score: 0.9958
MSE: 1.55
```



Experiment 5: K-Means Clustering & Elbow Method

Aim: Group dataset into optimal K clusters using K-Means and Elbow Curve analysis.

Complete Executable Python Source Code

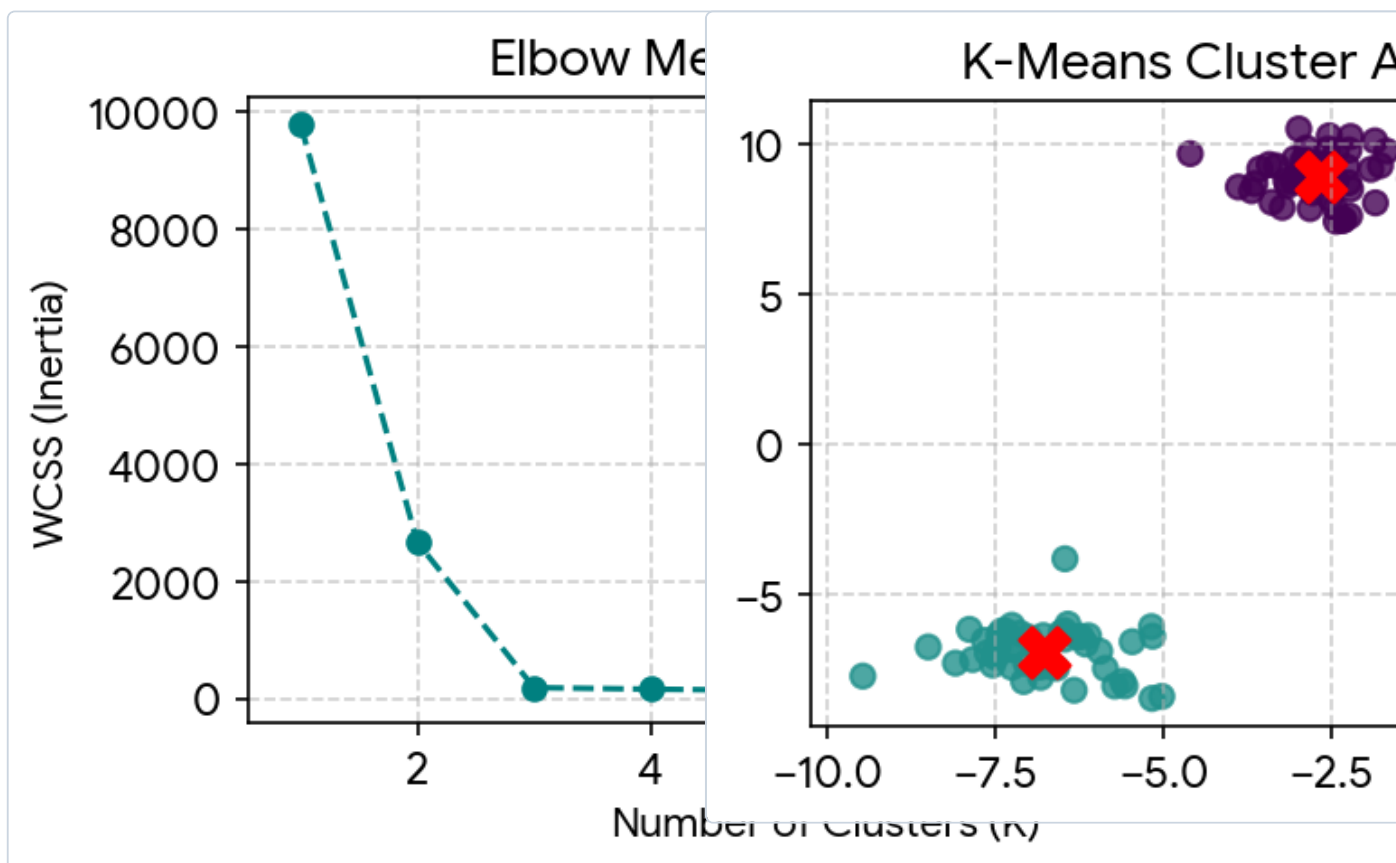
```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=150, centers=3, cluster_std=0.8, random_state=42)

# Elbow Method
wcss = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X)
    wcss.append(km.inertia_)

# Fit Final K-Means Model
km = KMeans(n_clusters=3, random_state=42, n_init=10)
y_km = km.fit_predict(X)
centroids = km.cluster_centers_
```

Execution Results & Visualizations



```
Calculated Cluster Centroids (K=3):
[[-2.66301205  8.9251602 ]
 [-6.77912214 -6.88758091]
 [ 4.59523718  2.10914946]]
```

Experiment 6: Hierarchical Agglomerative Clustering

Aim: Perform Hierarchical Agglomerative Clustering and plot Dendrogram diagram.

Complete Executable Python Source Code

```
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering
from sklearn.datasets import make_blobs

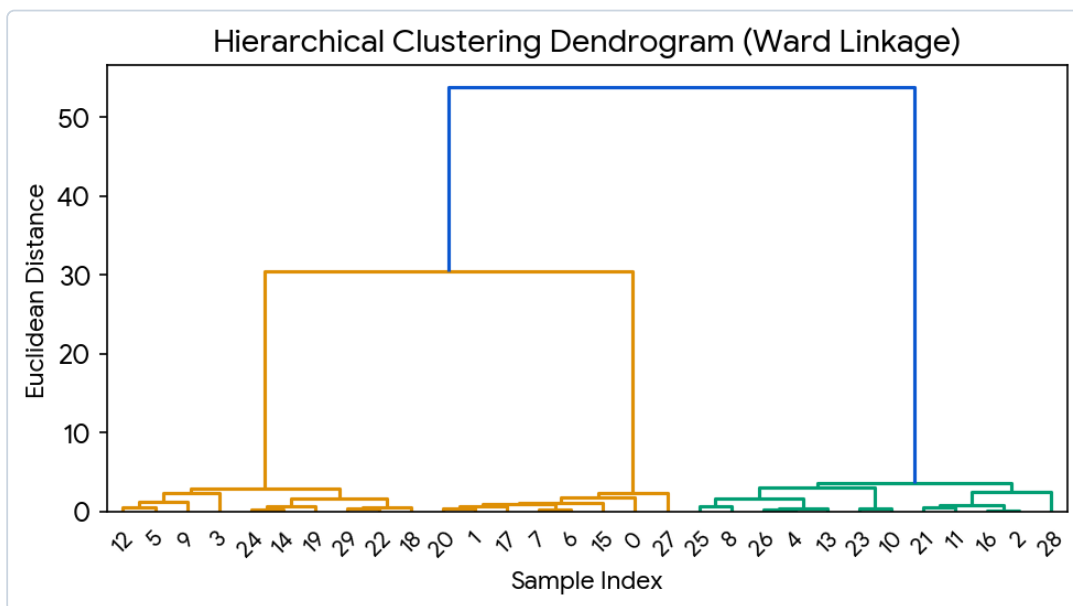
X, _ = make_blobs(n_samples=30, centers=3, cluster_std=0.8, random_state=42)

# Compute Linkage Matrix
linked = linkage(X, method='ward')

# Model Fitting
hc = AgglomerativeClustering(n_clusters=3, metric='euclidean', linkage='ward')
labels = hc.fit_predict(X)

# Dendrogram Plot
plt.figure(figsize=(8, 4))
dendrogram(linked, orientation='top', distance_sort='descending')
plt.title('Hierarchical Clustering Dendrogram')
plt.show()
```

Execution Result & Dendrogram Diagram



Experiment 7: Association Rule Mining (Apriori Algorithm)

Aim: Find frequent itemsets and association rules using Support, Confidence, and Lift.

Complete Executable Python Source Code

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

dataset = [
    ['Milk', 'Bread', 'Butter'],
    ['Bread', 'Diaper', 'Beer', 'Eggs'],
    ['Milk', 'Diaper', 'Beer', 'Cola'],
    ['Bread', 'Milk', 'Diaper', 'Beer'],
    ['Bread', 'Milk', 'Butter', 'Cola']
]

te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)

frequent_itemsets = apriori(df, min_support=0.4, use_colnames=True)
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.6)
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

Execution Result (Mined Rules Output Table)

Antecedent	Consequent	Support(A)	Support(B)	Support(A->B)	Confidence	Lift
Butter	Bread	0.4	0.8	0.4	1.000000	1.250000
Bread	Milk	0.8	0.8	0.6	0.750000	0.937500
Milk	Bread	0.8	0.8	0.6	0.750000	0.937500
Butter	Milk	0.4	0.8	0.4	1.000000	1.250000
Beer	Bread	0.6	0.8	0.4	0.666667	0.833333
Beer	Diaper	0.6	0.6	0.6	1.000000	1.666667
Diaper	Beer	0.6	0.6	0.6	1.000000	1.666667
Diaper	Bread	0.6	0.8	0.4	0.666667	0.833333
Beer	Milk	0.6	0.8	0.4	0.666667	0.833333
Cola	Milk	0.4	0.8	0.4	1.000000	1.250000
Diaper	Milk	0.6	0.8	0.4	0.666667	0.833333

Viva Q&A:

• **Q: What is the Apriori Property?**

A: All non-empty subsets of a frequent itemset must also be frequent.